

Chapter 1. Hello Transformers

In 2017, researchers at Google published a paper that proposed a novel neural network architecture for sequence modeling.¹ Dubbed the *Transformer*, this architecture outperformed recurrent neural networks (RNNs) on machine translation tasks, both in terms of translation quality and training cost.

In parallel, an effective transfer learning method called ULMFiT showed that training long short-term memory (LSTM) networks on a very large and diverse corpus could produce state-of-the-art text classifiers with little labeled data.²

These advances were the catalysts for two of today's most well-known transformers: the Generative Pretrained Transformer (GPT)³ and Bidirectional Encoder Representations from Transformers (BERT).⁴ By combining the Transformer architecture with unsupervised learning, these models removed the need to train task-specific architectures from scratch and broke almost every benchmark in NLP by a significant margin. Since the release of GPT and BERT, a zoo of transformer models has emerged; a timeline of the most prominent entries is shown in Figure 1-1.

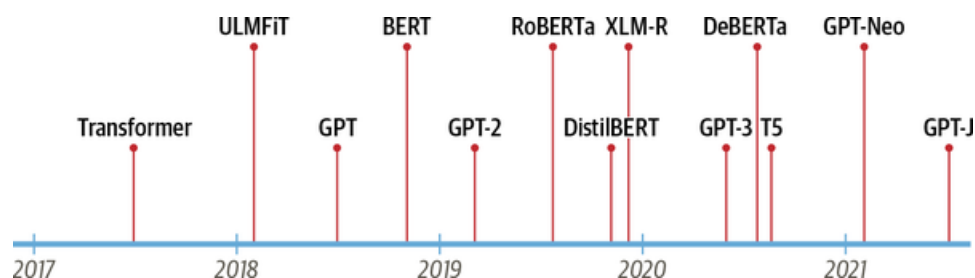


Figure 1-1. The transformers timeline

But we're getting ahead of ourselves. To understand what is novel about transformers, we first need to explain:

- The encoder-decoder framework
- Attention mechanisms
- Transfer learning

In this chapter we'll introduce the core concepts that underlie the pervasiveness of transformers, take a tour of some of the tasks that they excel at, and conclude with a look at the Hugging Face ecosystem of tools and libraries.

Let's start by exploring the encoder-decoder framework and the architectures that preceded the rise of transformers.

The Encoder-Decoder Framework

Prior to transformers, recurrent architectures such as LSTMs were the state of the art in NLP. These architectures contain a feedback loop in the network connections that allows information to propagate from one step to another, making them ideal for modeling sequential data like text. As illustrated on the left side of **Figure 1-2**, an RNN receives some input (which could be a word or character), feeds it through the network, and outputs a vector called the *hidden state*. At the same time, the model feeds some information back to itself through the feedback loop, which it can then use in the next step. This can be more clearly seen if we “unroll” the loop as shown on the right side of **Figure 1-2**: the RNN passes information about its state at each step to the next operation in the sequence. This allows an RNN to keep track of information from previous steps, and use it for its output predictions.

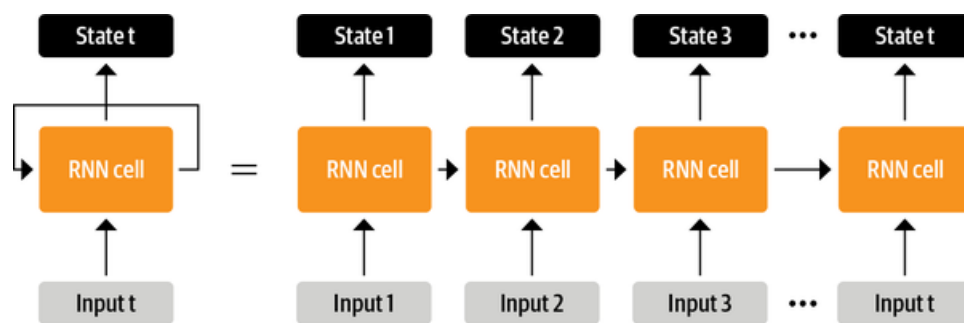


Figure 1-2. Unrolling an RNN in time

These architectures were (and continue to be) widely used for NLP tasks, speech processing, and time series. You can find a wonderful exposition of their capabilities in Andrej Karpathy’s blog post, “**The Unreasonable Effectiveness of Recurrent Neural Networks**”.

One area where RNNs played an important role was in the development of machine translation systems, where the objective is to map a sequence of words in one language to another. This kind of task is usually tackled with an *encoder-decoder* or *sequence-to-sequence* architecture,⁵ which is well suited for situations where the input and output are both sequences of arbitrary length. The job of the encoder is to encode the information from the input sequence into a numerical representation that is often called the *last hidden state*. This state is then passed to the decoder, which generates the output sequence.

In general, the encoder and decoder components can be any kind of neural network architecture that can model sequences. This is illustrated for a pair of RNNs in **Figure 1-3**, where the English sentence “Transformers are great!” is encoded as a hidden state vector that is then decoded to produce the German translation “Transformer sind grossartig!” The input words are fed sequentially through the encoder and the output words are generated one at a time, from top to bottom.

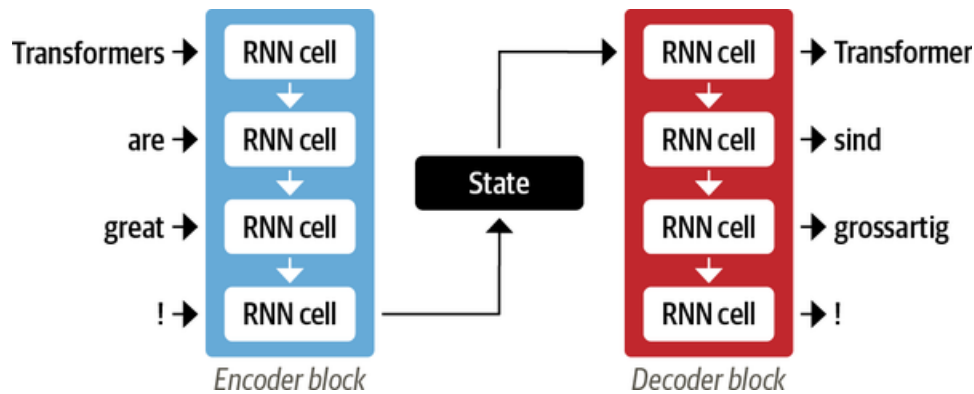


Figure 1-3. An encoder-decoder architecture with a pair of RNNs (in general, there are many more recurrent layers than those shown here)

Although elegant in its simplicity, one weakness of this architecture is that the final hidden state of the encoder creates an *information bottleneck*: it has to represent the meaning of the whole input sequence because this is all the decoder has access to when generating the output. This is especially challenging for long sequences, where information at the start of the sequence might be lost in the process of compressing everything to a single, fixed representation.

Fortunately, there is a way out of this bottleneck by allowing the decoder to have access to all of the encoder's hidden states. The general mechanism for this is called *attention*,⁶ and it is a key component in many modern neural network architectures. Understanding how attention was developed for RNNs will put us in good shape to understand one of the main building blocks of the Transformer architecture. Let's take a deeper look.

Attention Mechanisms

The main idea behind attention is that instead of producing a single hidden state for the input sequence, the encoder outputs a hidden state at each step that the decoder can access. However, using all the states at the same time would create a huge input for the decoder, so some mechanism is needed to prioritize which states to use. This is where attention comes in: it lets the decoder assign a different amount of weight, or "attention," to each of the encoder states at every decoding timestep. This process is illustrated in Figure 1-4, where the role of attention is shown for predicting the third token in the output sequence.

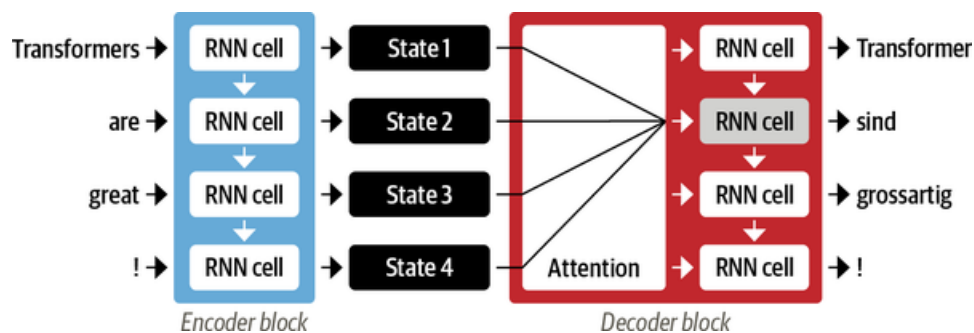


Figure 1-4. An encoder-decoder architecture with an attention mechanism for a pair of RNNs

By focusing on which input tokens are most relevant at each timestep, these attention-based models are able to learn nontrivial alignments between the words in a generated translation and those in a source sentence. For example, [Figure 1-5](#) visualizes the attention weights for an English to French translation model, where each pixel denotes a weight. The figure shows how the decoder is able to correctly align the words “zone” and “Area”, which are ordered differently in the two languages.

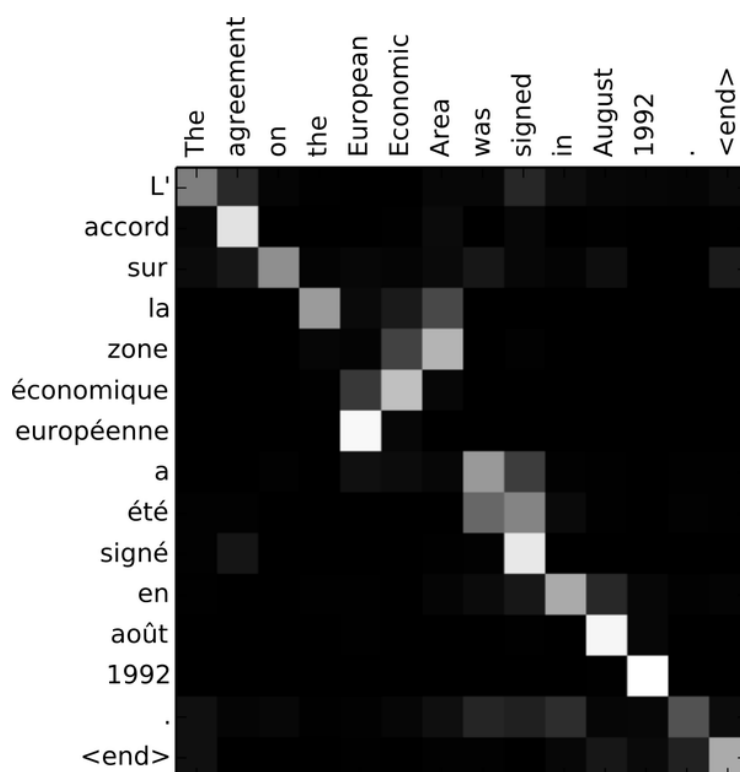


Figure 1-5. RNN encoder-decoder alignment of words in English and the generated translation in French (courtesy of Dzmitry Bahdanau)

Although attention enabled the production of much better translations, there was still a major shortcoming with using recurrent models for the encoder and decoder: the computations are inherently sequential and cannot be parallelized across the input sequence.

With the transformer, a new modeling paradigm was introduced: dispense with recurrence altogether, and instead rely entirely on a special form of attention called *self-attention*. We’ll cover self-attention in more detail in [Chapter 3](#), but the basic idea is to allow attention to operate on all the states in the *same layer* of the neural network. This is shown in [Figure 1-6](#), where both the encoder and the decoder have their own self-attention mechanisms, whose outputs are fed to feed-forward neural networks (FF NNs). This architecture can be trained much faster than recurrent models and paved the way for many of the recent breakthroughs in NLP.

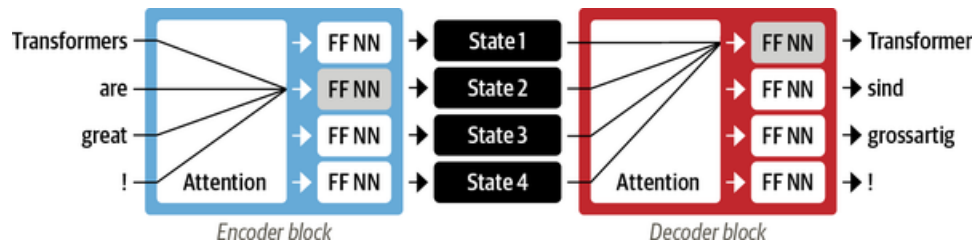


Figure 1-6. Encoder-decoder architecture of the original Transformer

In the original Transformer paper, the translation model was trained from scratch on a large corpus of sentence pairs in various languages. However, in many practical applications of NLP we do not have access to large amounts of labeled text data to train our models on. A final piece was missing to get the transformer revolution started: **transfer learning**.

Transfer Learning in NLP

It is nowadays common practice in computer vision to use transfer learning to train a **convolutional neural network like ResNet on one task, and then adapt it to or *fine-tune* it on a new task**. This allows the network to make use of the knowledge learned from the original task. Architecturally, this involves splitting the model into of a *body* and a *head*, where the head is a task-specific network. During training, the weights of the body learn broad features of the source domain, and these weights are used to initialize a new model for the new task.⁷ Compared to traditional supervised learning, this approach typically produces high-quality models that can be trained much more efficiently on a variety of downstream tasks, and with much less labeled data. A comparison of the two approaches is shown in **Figure 1-7**.

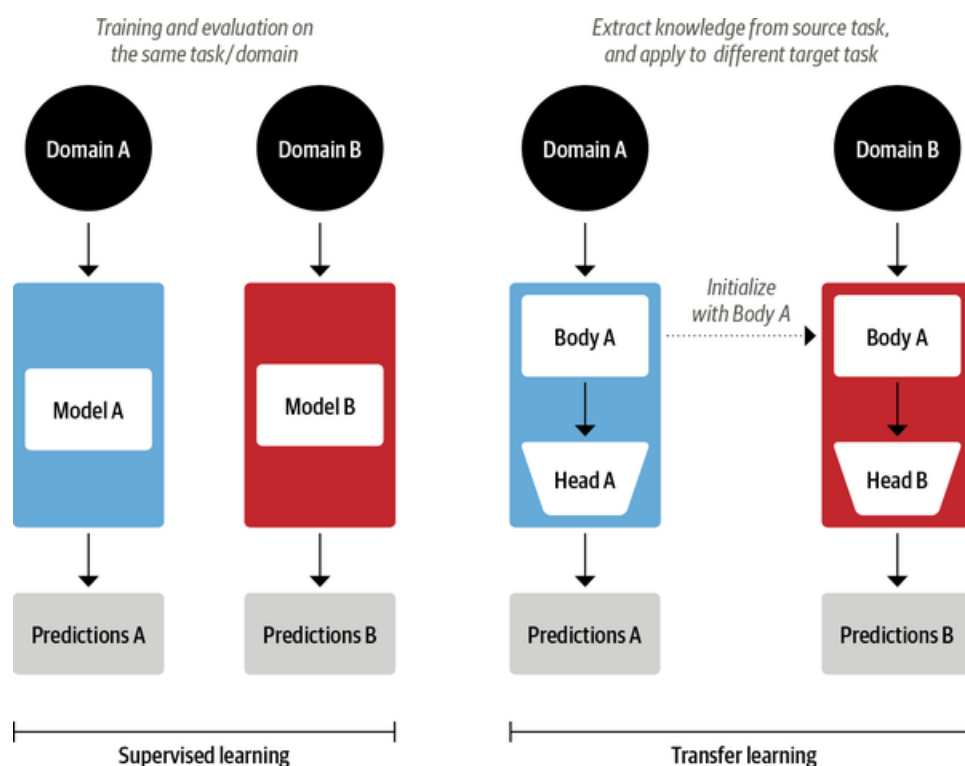


Figure 1-7. Comparison of traditional supervised learning (left) and transfer learning (right)

In computer vision, the models are first trained on large-scale datasets such as **ImageNet**, which contain millions of images. This process is called *pretraining* and its main purpose is to teach the models the basic features of images, such as edges or colors. These pretrained models can then be fine-tuned on a downstream task such as classifying flower species with a relatively small number of labeled examples (usually a few hundred per class). Fine-tuned models typically achieve a higher accuracy than supervised models trained from scratch on the same amount of labeled data.

Although transfer learning became the standard approach in computer vision, for many years it was not clear what the analogous pretraining process was for NLP. As a result, NLP applications typically required large amounts of labeled data to achieve high performance. And even then, that performance did not compare to what was achieved in the vision domain.

In 2017 and 2018, several research groups proposed new approaches that finally made transfer learning work for NLP. It started with an insight from researchers at OpenAI who obtained strong performance on a sentiment classification task by using features extracted from unsupervised pretraining.⁸ This was followed by **ULMFiT**, which introduced a general framework to adapt pretrained LSTM models for various tasks.⁹

As illustrated in **Figure 1-8**, ULMFiT involves three main steps:

Pretraining

The initial training objective is quite simple: predict the next word based on the previous words. This task is referred to as *language modeling*. The elegance of this approach lies in

the fact that no labeled data is required, and one can make use of abundantly available text from sources such as Wikipedia.¹⁰

Domain adaptation

Once the language model is pretrained on a large-scale corpus, the next step is to adapt it to the in-domain corpus (e.g., from Wikipedia to the IMDb corpus of movie reviews, as in Figure 1-8). This stage still uses language modeling, but now the model has to predict the next word in the target corpus.

Fine-tuning

In this step, the language model is fine-tuned with a classification layer for the target task (e.g., classifying the sentiment of movie reviews in Figure 1-8).

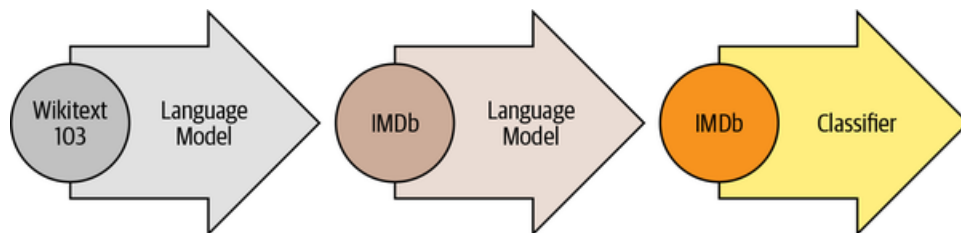


Figure 1-8. The ULMFiT process (courtesy of Jeremy Howard)

By introducing a viable framework for pretraining and transfer learning in NLP, ULMFiT provided the missing piece to make transformers take off. In 2018, two transformers were released that combined self-attention with transfer learning:

GPT

Uses only the decoder part of the Transformer architecture, and the same language modeling approach as ULMFiT. GPT was pretrained on the BookCorpus,¹¹ which consists of 7,000 unpublished books from a variety of genres including Adventure, Fantasy, and Romance.

BERT

Uses the encoder part of the Transformer architecture, and a special form of language modeling called *masked language modeling*. The objective of masked language modeling is to predict randomly masked words in a text. For example, given a sentence like “I looked at my [MASK] and saw that [MASK] was late.” the model needs to predict the most likely candidates for the masked words that are denoted by [MASK]. BERT was pretrained on the BookCorpus and English Wikipedia.

GPT and BERT set a new state of the art across a variety of NLP benchmarks and ushered in the age of transformers.

However, with different research labs releasing their models in incompatible frameworks (PyTorch or TensorFlow), it wasn't always easy for NLP practitioners to port these models to their own applications. With the release of 🤖 **Transformers**, a unified API across more than 50 architectures was progressively built. This library catalyzed the explosion of research into transformers and quickly trickled down to NLP practitioners, making it easy to integrate these models into many real-life applications today. Let's have a look!

Hugging Face Transformers: Bridging the Gap

Applying a novel machine learning architecture to a new task can be a complex undertaking, and usually involves the following steps:

1. Implement the model architecture in code, typically based on PyTorch or TensorFlow.
2. Load the pretrained weights (if available) from a server.
3. Preprocess the inputs, pass them through the model, and apply some task-specific postprocessing.
4. Implement dataloaders and define loss functions and optimizers to train the model.

Each of these steps requires custom logic for each model and task. Traditionally (but not always!), when research groups publish a new article, they will also release the code along with the model weights. However, this code is rarely standardized and often requires days of engineering to adapt to new use cases.

This is where 🤖 Transformers comes to the NLP practitioner's rescue! It provides a standardized interface to a wide range of transformer models as well as code and tools to adapt these models to new use cases. The library currently supports three major deep learning frameworks (PyTorch, TensorFlow, and JAX) and allows you to easily switch between them. In addition, it provides task-specific heads so you can easily fine-tune transformers on downstream tasks such as text classification, named entity recognition, and question answering. This reduces the time it takes a practitioner to train and test a handful of models from a week to a single afternoon!

You'll see this for yourself in the next section, where we show that with just a few lines of code, 🤖 Transformers can be applied to tackle some of the most common NLP applications that you're likely to encounter in the wild.

A Tour of Transformer Applications

Every NLP task starts with a piece of text, like the following made-up customer feedback about a certain online order:

```
text = """Dear Amazon, last week I ordered an Optimus Prime action figure  
from your online store in Germany. Unfortunately, when I opened the package,
```



```
I discovered to my horror that I had been sent an action figure of Megatron instead! As a lifelong enemy of the Decepticons, I hope you can understand my dilemma. To resolve the issue, I demand an exchange of Megatron for the Optimus Prime figure I ordered. Enclosed are copies of my records concerning this purchase. I expect to hear from you soon. Sincerely, Bumblebee."""
```

Depending on your application, the text you're working with could be a legal contract, a product description, or something else entirely. In the case of customer feedback, you would probably like to know whether the feedback is positive or negative. This task is called *sentiment analysis* and is part of the broader topic of *text classification* that we'll explore in **Chapter 2**. For now, let's have a look at what it takes to extract the sentiment from our piece of text using 🤖 Transformers.

Text Classification

As we'll see in later chapters, 🤖 Transformers has a layered API that allows you to interact with the library at various levels of abstraction. In this chapter we'll start with *pipelines*, which abstract away all the steps needed to convert raw text into a set of predictions from a fine-tuned model.

In 🤖 Transformers, we instantiate a pipeline by calling the `pipeline()` function and providing the name of the task we are interested in:

```
from transformers import pipeline

classifier = pipeline("text-classification")
```

The first time you run this code you'll see a few progress bars appear because the pipeline automatically downloads the model weights from the **Hugging Face Hub**. The second time you instantiate the pipeline, the library will notice that you've already downloaded the weights and will use the cached version instead. By default, the `text-classification` pipeline uses a model that's designed for sentiment analysis, but it also supports multiclass and multilabel classification.

Now that we have our pipeline, let's generate some predictions! Each pipeline takes a string of text (or a list of strings) as input and returns a list of predictions. Each prediction is a Python dictionary, so we can use Pandas to display them nicely as a `DataFrame`:

```
import pandas as pd

outputs = classifier(text)
pd.DataFrame(outputs)
```

	label	score
0	NEGATIVE	0.901546

In this case the model is very confident that the text has a negative sentiment, which makes sense given that we're dealing with a complaint from an angry customer! Note that for sentiment analysis tasks the pipeline only returns one of the `POSITIVE` or `NEGATIVE` labels, since the other can be inferred by computing `1 - score`.

Let's now take a look at another common task, identifying named entities in text.

Named Entity Recognition

Predicting the sentiment of customer feedback is a good first step, but you often want to know if the feedback was about a particular item or service. In NLP, real-world objects like products, places, and people are called *named entities*, and extracting them from text is called *named entity recognition* (NER). We can apply NER by loading the corresponding pipeline and feeding our customer review to it:

```
ner_tagger = pipeline("ner", aggregation_strategy="simple")
outputs = ner_tagger(text)
pd.DataFrame(outputs)
```

	entity_group	score	word	start	end
0	ORG	0.879010	Amazon	5	11
1	MISC	0.990859	Optimus Prime	36	49
2	LOC	0.999755	Germany	90	97
3	MISC	0.556569	Mega	208	212
4	PER	0.590256	##tron	212	216
5	ORG	0.669692	Decept	253	259
6	MISC	0.498350	##icons	259	264
7	MISC	0.775361	Megatron	350	358
8	MISC	0.987854	Optimus Prime	367	380
9	PER	0.812096	Bumblebee	502	511

You can see that the pipeline detected all the entities and also assigned a category such as `ORG` (organization), `LOC` (location), or `PER` (person) to each of them. Here we used the `aggregation_strategy` argument to group the words according to the model's predictions. For example, the entity "Optimus Prime" is composed of two words, but is assigned a single category: `MISC` (miscellaneous). The scores tell us how confident the model was about the entities it identified. We can see that it was least confident about "Decepticons" and the first occurrence of "Megatron", both of which it failed to group as a single entity.

NOTE

See those weird hash symbols (#) in the `word` column in the previous table? These are produced by the model's `tokenizer`, which splits words into atomic units called *tokens*. You'll learn all about tokenization in [Chapter 2](#).

Extracting all the named entities in a text is nice, but sometimes we would like to ask more targeted questions. This is where we can use *question answering*.

Question Answering

In question answering, we provide the model with a passage of text called the *context*, along with a question whose answer we'd like to extract. The model then returns the span of text corresponding to the answer. Let's see what we get when we ask a specific question about our customer feedback:

```
reader = pipeline("question-answering")
question = "What does the customer want?"
outputs = reader(question=question, context=text)
pd.DataFrame([outputs])
```

	score	start	end	answer
0	0.631291	335	358	an exchange of Megatron

We can see that along with the answer, the pipeline also returned `start` and `end` integers that correspond to the character indices where the answer span was found (just like with NER tagging). There are several flavors of question answering that we will investigate in [Chapter 7](#), but this particular kind is called *extractive question answering* because the answer is extracted directly from the text.

With this approach you can read and extract relevant information quickly from a customer's feedback. But what if you get a mountain of long-winded complaints and you don't have the

time to read them all? Let's see if a summarization model can help!

Summarization

The goal of text summarization is to take a long text as input and generate a short version with all the relevant facts. This is a much more complicated task than the previous ones since it requires the model to *generate* coherent text. In what should be a familiar pattern by now, we can instantiate a summarization pipeline as follows:

```
summarizer = pipeline("summarization")
outputs = summarizer(text, max_length=45, clean_up_tokenization_spaces=True)
print(outputs[0]['summary_text'])
```

```
Bumblebee ordered an Optimus Prime action figure from your online store in
Germany. Unfortunately, when I opened the package, I discovered to my horror
that I had been sent an action figure of Megatron instead.
```

This summary isn't too bad! Although parts of the original text have been copied, the model was able to capture the essence of the problem and correctly identify that "Bumblebee" (which appeared at the end) was the author of the complaint. In this example you can also see that we passed some keyword arguments like `max_length` and `clean_up_tokenization_spaces` to the pipeline; these allow us to tweak the outputs at runtime.

But what happens when you get feedback that is in a language you don't understand? You could use Google Translate, or you can use your very own transformer to translate it for you!

Translation

Like summarization, translation is a task where the output consists of generated text. Let's use a translation pipeline to translate an English text to German:

```
translator = pipeline("translation_en_to_de",
                      model="Helsinki-NLP/opus-mt-en-de")
outputs = translator(text, clean_up_tokenization_spaces=True, min_length=100)
print(outputs[0]['translation_text'])
```

```
Sehr geehrter Amazon, letzte Woche habe ich eine Optimus Prime Action Figur aus
Ihrem Online-Shop in Deutschland bestellt. Leider, als ich das Paket öffnete,
entdeckte ich zu meinem Entsetzen, dass ich stattdessen eine Action Figur von
Megatron geschickt worden war! Als lebenslanger Feind der Decepticons, Ich
hoffe, Sie können mein Dilemma verstehen. Um das Problem zu lösen, Ich fordere
einen Austausch von Megatron für die Optimus Prime Figur habe ich bestellt.
Anbei sind Kopien meiner Aufzeichnungen über diesen Kauf. Ich erwarte, bald von
Ihnen zu hören. Aufrichtig, Bumblebee.
```

Again, the model produced a very good translation that correctly uses German's formal pronouns, like "Ihrem" and "Sie." Here we've also shown how you can override the default model in the pipeline to pick the best one for your application—and you can find models for

thousands of language pairs on the Hugging Face Hub. Before we take a step back and look at the whole Hugging Face ecosystem, let's examine one last application.

Text Generation

Let's say you would like to be able to provide faster replies to customer feedback by having access to an autocomplete function. With a text generation model you can do this as follows:

```
generator = pipeline("text-generation")
response = "Dear Bumblebee, I am sorry to hear that your order was mixed up."
prompt = text + "\n\nCustomer service response:\n" + response
outputs = generator(prompt, max_length=200)
print(outputs[0]['generated_text'])
```

Dear Amazon, last week I ordered an Optimus Prime action figure from your online store in Germany. Unfortunately, when I opened the package, I discovered to my horror that I had been sent an action figure of Megatron instead! As a lifelong enemy of the Decepticons, I hope you can understand my dilemma. To resolve the issue, I demand an exchange of Megatron for the Optimus Prime figure I ordered. Enclosed are copies of my records concerning this purchase. I expect to hear from you soon. Sincerely, Bumblebee.

Customer service response:

Dear Bumblebee, I am sorry to hear that your order was mixed up. The order was completely mislabeled, which is very common in our online store, but I can appreciate it because it was my understanding from this site and our customer service of the previous day that your order was not made correct in our mind and that we are in a process of resolving this matter. We can assure you that your order

OK, maybe we wouldn't want to use this completion to calm Bumblebee down, but you get the general idea.

Now that you've seen a few cool applications of transformer models, you might be wondering where the training happens. All of the models that we've used in this chapter are publicly available and already fine-tuned for the task at hand. In general, however, you'll want to fine-tune models on your own data, and in the following chapters you will learn how to do just that.

But training a model is just a small piece of any NLP project—being able to efficiently process data, share results with colleagues, and make your work reproducible are key components too. Fortunately, 🤖 Transformers is surrounded by a big ecosystem of useful tools that support much of the modern machine learning workflow. Let's take a look.

The Hugging Face Ecosystem

What started with 🤖 Transformers has quickly grown into a whole ecosystem consisting of many libraries and tools to accelerate your NLP and machine learning projects. The Hugging Face ecosystem consists of mainly two parts: a family of libraries and the Hub, as shown in [Figure 1-9](#). The libraries provide the code while the Hub provides the pretrained model weights, datasets, scripts for the evaluation metrics, and more. In this section we'll have a brief

look at the various components. We'll skip 🤗 Transformers, as we've already discussed it and we will see a lot more of it throughout the course of the book.

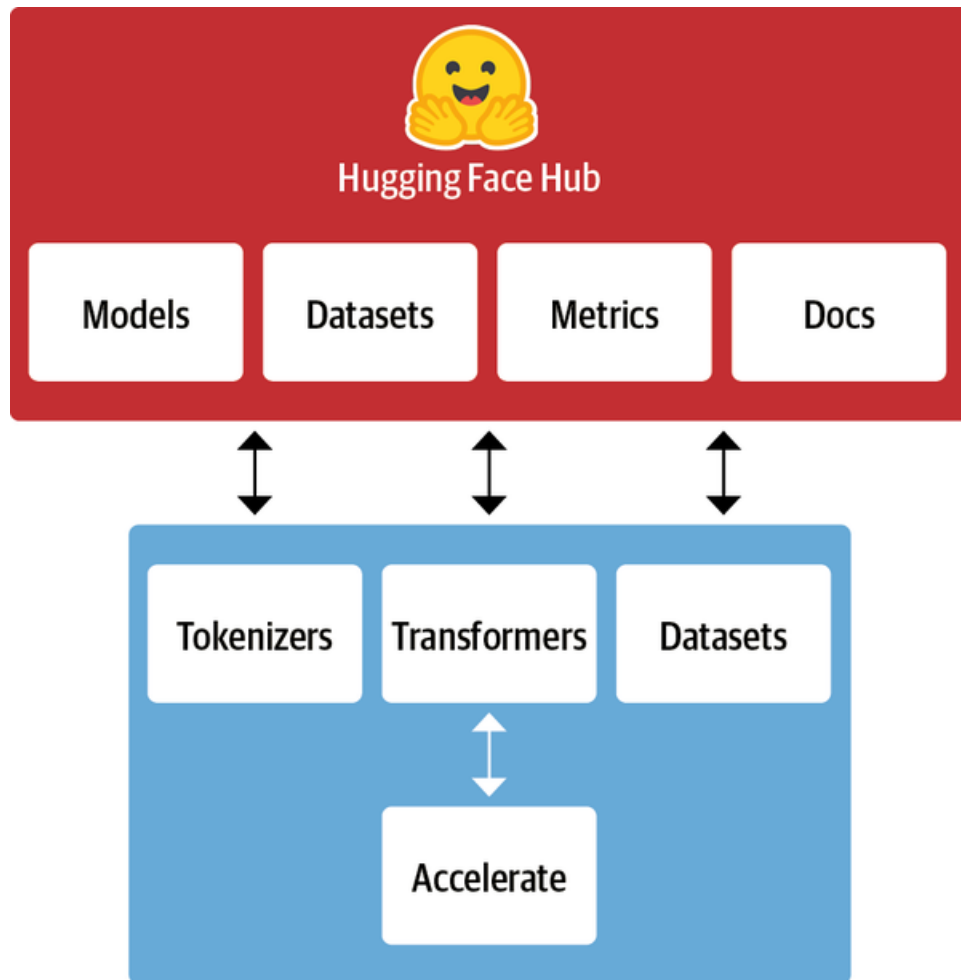


Figure 1-9. An overview of the Hugging Face ecosystem

The Hugging Face Hub

As outlined earlier, transfer learning is one of the key factors driving the success of transformers because it makes it possible to reuse pretrained models for new tasks. Consequently, it is crucial to be able to load pretrained models quickly and run experiments with them.

The Hugging Face Hub hosts over 20,000 freely available models. As shown in [Figure 1-10](#), there are filters for tasks, frameworks, datasets, and more that are designed to help you navigate the Hub and quickly find promising candidates. As we've seen with the pipelines, loading a promising model in your code is then literally just one line of code away. This makes experimenting with a wide range of models simple, and allows you to focus on the domain-specific parts of your project.

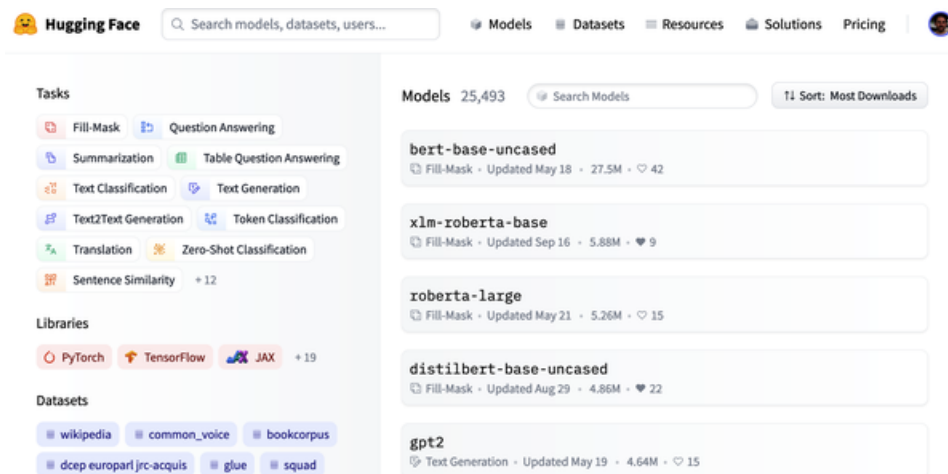


Figure 1-10. The Models page of the Hugging Face Hub, showing filters on the left and a list of models on the right

In addition to model weights, the Hub also hosts datasets and scripts for computing metrics, which let you reproduce published results or leverage additional data for your application.

The Hub also provides *model* and *dataset cards* to document the contents of models and datasets and help you make an informed decision about whether they're the right ones for you. One of the coolest features of the Hub is that you can try out any model directly through the various task-specific interactive widgets as shown in Figure 1-11.

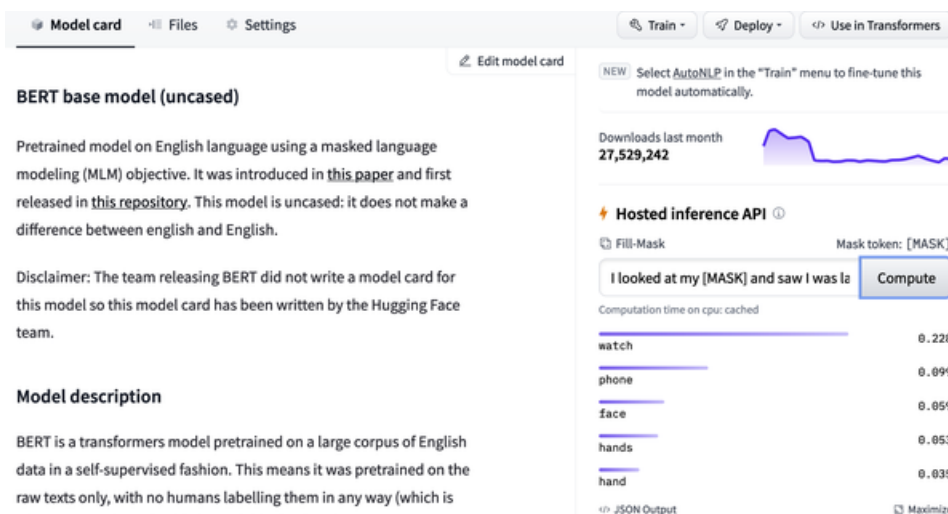


Figure 1-11. An example model card from the Hugging Face Hub: the inference widget, which allows you to interact with the model, is shown on the right

Let's continue our tour with 🧑 Tokenizers.

NOTE

PyTorch and TensorFlow also offer hubs of their own and are worth checking out if a particular model or dataset is not available on the Hugging Face Hub.

Hugging Face Tokenizers

Behind each of the pipeline examples that we've seen in this chapter is a tokenization step that splits the raw text into smaller pieces called tokens. We'll see how this works in detail in [Chapter 2](#), but for now it's enough to understand that tokens may be words, parts of words, or just characters like punctuation. Transformer models are trained on numerical representations of these tokens, so getting this step right is pretty important for the whole NLP project!

🤗 **Tokenizers** provides many tokenization strategies and is extremely fast at tokenizing text thanks to its Rust backend.¹² It also takes care of all the pre- and postprocessing steps, such as normalizing the inputs and transforming the model outputs to the required format. With 🤗 **Tokenizers**, we can load a tokenizer in the same way we can load pretrained model weights with 🤗 **Transformers**.

We need a dataset and metrics to train and evaluate models, so let's take a look at 🤗 **Datasets**, which is in charge of that aspect.

Hugging Face Datasets

Loading, processing, and storing datasets can be a cumbersome process, especially when the datasets get too large to fit in your laptop's RAM. In addition, you usually need to implement various scripts to download the data and transform it into a standard format.

🤗 **Datasets** simplifies this process by providing a standard interface for thousands of datasets that can be found on the [Hub](#). It also provides smart caching (so you don't have to redo your preprocessing each time you run your code) and avoids RAM limitations by leveraging a special mechanism called *memory mapping* that stores the contents of a file in virtual memory and enables multiple processes to modify a file more efficiently. The library is also interoperable with popular frameworks like Pandas and NumPy, so you don't have to leave the comfort of your favorite data wrangling tools.

Having a good dataset and powerful model is worthless, however, if you can't reliably measure the performance. Unfortunately, classic NLP metrics come with many different implementations that can vary slightly and lead to deceptive results. By providing the scripts for many metrics, 🤗 **Datasets** helps make experiments more reproducible and the results more trustworthy.

With the 🤗 **Transformers**, 🤗 **Tokenizers**, and 🤗 **Datasets** libraries we have everything we need to train our very own transformer models! However, as we'll see in [Chapter 10](#) there are situations where we need fine-grained control over the training loop. That's where the last library of the ecosystem comes into play: 🤗 **Accelerate**.

Hugging Face Accelerate

If you've ever had to write your own training script in PyTorch, chances are that you've had some headaches when trying to port the code that runs on your laptop to the code that runs on your organization's cluster. 🤗 **Accelerate** adds a layer of abstraction to your normal training

loops that takes care of all the custom logic necessary for the training infrastructure. This literally accelerates your workflow by simplifying the change of infrastructure when necessary.

This sums up the core components of Hugging Face’s open source ecosystem. But before wrapping up this chapter, let’s take a look at a few of the common challenges that come with trying to deploy transformers in the real world.

Main Challenges with Transformers

In this chapter we’ve gotten a glimpse of the wide range of NLP tasks that can be tackled with transformer models. Reading the media headlines, it can sometimes sound like their capabilities are limitless. However, despite their usefulness, transformers are far from being a silver bullet. Here are a few challenges associated with them that we will explore throughout the book:

Language

NLP research is dominated by the English language. There are several models for other languages, but it is harder to find pretrained models for rare or low-resource languages. In [Chapter 4](#), we’ll explore multilingual transformers and their ability to perform zero-shot cross-lingual transfer.

Data availability

Although we can use transfer learning to dramatically reduce the amount of labeled training data our models need, it is still a lot compared to how much a human needs to perform the task. Tackling scenarios where you have little to no labeled data is the subject of [Chapter 9](#).

Working with long documents

Self-attention works extremely well on paragraph-long texts, but it becomes very expensive when we move to longer texts like whole documents. Approaches to mitigate this are discussed in [Chapter 11](#).

Opacity

As with other deep learning models, transformers are to a large extent opaque. It is hard or impossible to unravel “why” a model made a certain prediction. This is an especially hard challenge when these models are deployed to make critical decisions. We’ll explore some ways to probe the errors of transformer models in [Chapters 2 and 4](#).

Bias

Transformer models are predominantly pretrained on text data from the internet. This imprints all the biases that are present in the data into the models. Making sure that these are neither racist, sexist, or worse is a challenging task. We discuss some of these issues in more detail in [Chapter 10](#).

Although daunting, many of these challenges can be overcome. As well as in the specific chapters mentioned, we will touch on these topics in almost every chapter ahead.

Conclusion

Hopefully, by now you are excited to learn how to start training and integrating these versatile models into your own applications! You've seen in this chapter that with just a few lines of code you can use state-of-the-art models for classification, named entity recognition, question answering, translation, and summarization, but this is really just the "tip of the iceberg."

In the following chapters you will learn how to adapt transformers to a wide range of use cases, such as building a text classifier, or a lightweight model for production, or even training a language model from scratch. We'll be taking a hands-on approach, which means that for every concept covered there will be accompanying code that you can run on Google Colab or your own GPU machine.

Now that we're armed with the basic concepts behind transformers, it's time to get our hands dirty with our first application: text classification. That's the topic of the next chapter!

-
- 1 A. Vaswani et al., "[Attention Is All You Need](#)", (2017). This title was so catchy that no less than [50 follow-up papers](#) have included "all you need" in their titles!
 - 2 J. Howard and S. Ruder, "[Universal Language Model Fine-Tuning for Text Classification](#)", (2018).
 - 3 A. Radford et al., "[Improving Language Understanding by Generative Pre-Training](#)", (2018).
 - 4 J. Devlin et al., "[BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding](#)", (2018).
 - 5 I. Sutskever, O. Vinyals, and Q.V. Le, "[Sequence to Sequence Learning with Neural Networks](#)", (2014).
 - 6 D. Bahdanau, K. Cho, and Y. Bengio, "[Neural Machine Translation by Jointly Learning to Align and Translate](#)", (2014).
 - 7 Weights are the learnable parameters of a neural network.
 - 8 A. Radford, R. Jozefowicz, and I. Sutskever, "[Learning to Generate Reviews and Discovering Sentiment](#)", (2017).
 - 9 A related work at this time was ELMo (Embeddings from Language Models), which showed how pretraining LSTMs could produce high-quality word embeddings for downstream tasks.
 - 10 This is more true for English than for most of the world's languages, where obtaining a large corpus of digitized text can be difficult. Finding ways to bridge this gap is an active area of NLP research and activism.
 - 11 Y. Zhu et al., "[Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books](#)", (2015).
 - 12 [Rust](#) is a high-performance programming language.

Chapter 2. Text Classification

Text classification is one of the most common tasks in NLP; it can be used for a broad range of applications, such as tagging customer feedback into categories or routing support tickets according to their language. Chances are that your email program’s spam filter is using text classification to protect your inbox from a deluge of unwanted junk!

Another common type of text classification is sentiment analysis, which (as we saw in [Chapter 1](#)) aims to identify the polarity of a given text. For example, a company like Tesla might analyze Twitter posts like the one in [Figure 2-1](#) to determine whether people like its new car roofs or not.



Figure 2-1. Analyzing Twitter content can yield useful feedback from customers (courtesy of Aditya Veluri)

Now imagine that you are a data scientist who needs to build a system that can automatically identify emotional states such as “anger” or “joy” that people express about your company’s product on Twitter. In this chapter, we’ll tackle this task using a variant of BERT called DistilBERT.¹ The main advantage of this model is that it achieves comparable performance to BERT, while being significantly smaller and more efficient. This enables us to train a classifier in a few minutes, and if you want to train a larger BERT model you can simply change the checkpoint of the pretrained model. A *checkpoint* corresponds to the set of weights that are loaded into a given transformer architecture.

This will also be our first encounter with three of the core libraries from the Hugging Face ecosystem: 🗃 Datasets, 🗣 Tokenizers, and 🧠 Transformers. As shown in [Figure 2-2](#), these libraries will allow us to quickly go from raw text to a fine-tuned model that can be used for

inference on new tweets. So, in the spirit of Optimus Prime, let's dive in, “transform, and roll out!”²

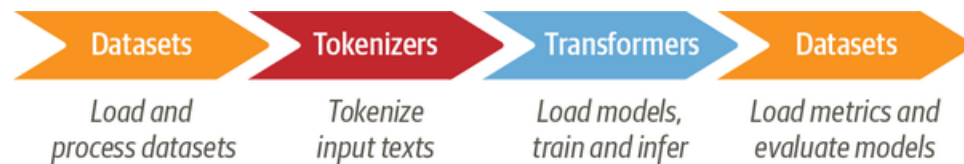


Figure 2-2. A typical pipeline for training transformer models with the 🧠 Datasets, 🧠 Tokenizers, and 🧠 Transformers libraries

The Dataset

To build our emotion detector we'll use a great dataset from an article that explored how emotions are represented in English Twitter messages.³ Unlike most sentiment analysis datasets that involve just “positive” and “negative” polarities, this dataset contains six basic emotions: anger, disgust, fear, joy, sadness, and surprise. Given a tweet, our task will be to train a model that can classify it into one of these emotions.

A First Look at Hugging Face Datasets

We will use 🧠 Datasets to download the data from the [Hugging Face Hub](#). We can use the `list_datasets()` function to see what datasets are available on the Hub:

```
from datasets import list_datasets

all_datasets = list_datasets()
print(f"There are {len(all_datasets)} datasets currently available on the Hub")
print(f"The first 10 are: {all_datasets[:10]}")
```

```
There are 1753 datasets currently available on the Hub
The first 10 are: ['acronym_identification', 'ade_corpus_v2', 'adversarial_qa',
'aeslc', 'afrikaans_ner_corpus', 'ag_news', 'ai2_arc', 'air_dialogue',
'ajgt_twitter_ar', 'allegro_reviews']
```

We see that each dataset is given a name, so let's load the emotion dataset with the `load_dataset()` function:

```
from datasets import load_dataset

emotions = load_dataset("emotion")
```

If we look inside our `emotions` object:

```
emotions

DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
```

```

        num_rows: 16000
    })
    validation: Dataset({
        features: ['text', 'label'],
        num_rows: 2000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 2000
    })
})

```

we see it is similar to a Python dictionary, with each key corresponding to a different split. And we can use the usual dictionary syntax to access an individual split:

```

train_ds = emotions["train"]
train_ds

Dataset({
  features: ['text', 'label'],
  num_rows: 16000
})

```

which returns an instance of the `Dataset` class. The `Dataset` object is one of the core data structures in 🍌 Datasets, and we'll be exploring many of its features throughout the course of this book. For starters, it behaves like an ordinary Python array or list, so we can query its length:

```

len(train_ds)

16000

```

or access a single example by its index:

```

train_ds[0]

{'label': 0, 'text': 'i didnt feel humiliated'}

```

Here we see that a single row is represented as a dictionary, where the keys correspond to the column names:

```

train_ds.column_names

['text', 'label']

```

and the values are the tweet and the emotion. This reflects the fact that 🍌 Datasets is based on *Apache Arrow*, which defines a typed columnar format that is more memory efficient than native Python. We can see what data types are being used under the hood by accessing the `features` attribute of a `Dataset` object:

```
print(train_ds.features)
```

```
{'text': Value(dtype='string', id=None), 'label': ClassLabel(num_classes=6,
names=['sadness', 'joy', 'love', 'anger', 'fear', 'surprise'], names_file=None,
id=None)}
```

In this case, the data type of the text column is string, while the label column is a special ClassLabel object that contains information about the class names and their mapping to integers. We can also access several rows with a slice:

```
print(train_ds[:5])
```

```
{'text': ['i didnt feel humiliated', 'i can go from feeling so hopeless to so
damned hopeful just from being around someone who cares and is awake', 'im
grabbing a minute to post i feel greedy wrong', 'i am ever feeling nostalgic
about the fireplace i will know that it is still on the property', 'i am feeling
grouchy'], 'label': [0, 0, 3, 2, 3]}
```

Note that in this case, the dictionary values are now lists instead of individual elements. We can also get the full column by name:

```
print(train_ds["text"][:5])
```

```
['i didnt feel humiliated', 'i can go from feeling so hopeless to so damned
hopeful just from being around someone who cares and is awake', 'im grabbing a
minute to post i feel greedy wrong', 'i am ever feeling nostalgic about the
fireplace i will know that it is still on the property', 'i am feeling grouchy']
```

Now that we've seen how to load and inspect data with 🍌 Datasets, let's do a few checks about the content of our tweets.

WHAT IF MY DATASET IS NOT ON THE HUB?

We'll be using the Hugging Face Hub to download datasets for most of the examples in this book. But in many cases, you'll find yourself working with data that is either stored on your laptop or on a remote server in your organization. 😊 Datasets provides several loading scripts to handle local and remote datasets. Examples for the most common data formats are shown in [Table 2-1](#).

*T
a
b
l
e
2
-
1
.
H
o
w

t
o
l
o
a
d
d
a
t
a
s
e
t
s
i
n
v
a
r
i
o
u
s
f
o
r
m
a
t
s*

Data format	Loading script	Example
CSV	csv	<code>load_dataset("csv", data_files="my_file.csv")</code>
Text	text	<code>load_dataset("text", data_files="my_file.txt")</code>
JSON	json	<code>load_dataset("json", data_files="my_file.jsonl")</code>

As you can see, for each data format, we just need to pass the relevant loading script to the `load_dataset()` function, along with a `data_files` argument that specifies the path or URL to one or more files. For example, the source files for the `emotion` dataset are actually hosted on Dropbox, so an alternative way to load the dataset is to first download one of the splits:

```
dataset_url = "https://www.dropbox.com/s/1pzkadvfffbqw6o/train.txt"
!wget {dataset_url}
```

If you're wondering why there's a `!` character in the preceding shell command, that's because we're running the commands in a Jupyter notebook. Simply remove the prefix if you want to download and unzip the dataset within a terminal. Now, if we peek at the first row of the `train.txt` file:

```
!head -n 1 train.txt

i didnt feel humiliated;sadness
```

we can see that here are no column headers and each tweet and emotion are separated by a semicolon. Nevertheless, this is quite similar to a CSV file, so we can load the dataset locally by using the `csv` script and pointing the `data_files` argument to the `train.txt` file:

```
emotions_local = load_dataset("csv", data_files="train.txt", sep=";",
                             names=["text", "label"])
```

Here we've also specified the type of delimiter and the names of the columns. An even simpler approach is to just point the `data_files` argument to the URL itself:

```
dataset_url = "https://www.dropbox.com/s/1pzkadvfffbqw6o/train.txt?dl=1"
emotions_remote = load_dataset("csv", data_files=dataset_url, sep=";",
                              names=["text", "label"])
```

which will automatically download and cache the dataset for you. As you can see, the `load_dataset()` function is very versatile. We recommend checking out the 🤗